

Handling Execution Overruns in Hard Real-Time Control Systems: Concepts and a Prototype Simulation Approach

Christian Percival
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany

Abstract—Hard real-time systems must complete critical tasks within strict timing constraints. Traditional schedulability analysis often uses worst-case execution times (WCETs) to guarantee safety, but this can lead to inefficient processor utilization when tasks normally execute much faster than their WCET. This problem is especially relevant in control systems with input-dependent execution times, such as visual tracking tasks. This paper studies the handling of execution overruns in hard real-time control systems, focusing on rate adaptation, bandwidth reservation, the Constant Bandwidth Server (CBS), and the hard-deadline CBS extension CBS_{hd}. In addition to the theoretical discussion, a software prototype is proposed for learning and demonstration purposes. The prototype uses a simplified image-processing task in which an object is first searched in a predicted window and, if not found, in progressively larger regions. This creates normal, overrun, and worst-case execution scenarios that can be used to compare WCET-based scheduling, normal-time scheduling, and simplified budget-based overrun handling.

I. INTRODUCTION

Real-time systems are computing systems where correctness depends not only on the logical result of a computation, but also on the time at which the result is produced. In hard real-time systems, missing a deadline can lead to severe consequences, especially in safety-critical control applications such as robotics, automotive systems, industrial automation, aerospace systems, and medical systems.

A common approach in hard real-time scheduling is to analyze each task using its worst-case execution time (WCET). This allows the system designer to check whether all tasks can meet their deadlines even under worst-case conditions. However, many real-time tasks do not normally execute for their WCET. Instead, they may have a short normal execution time and only occasionally require much longer computation.

This creates a conflict between predictability and efficiency. If all tasks are scheduled according to WCET, the system may be safe but inefficient. If tasks are scheduled according to normal execution time, processor utilization improves, but occasional execution overruns may cause deadline misses.

An execution overrun occurs when a task executes longer than its expected or reserved execution budget. This is different from simply finishing earlier than WCET. Finishing earlier than WCET means that reserved processor time is unused. An overrun means that a task needs more time than the scheduler expected for that job.

The main reference paper by Caccamo, Buttazzo, and Sha addresses this problem for hard real-time control systems. Their work is motivated by control applications such as visual tracking, where a task normally finds an object quickly in a small predicted search window, but occasionally needs to search a larger area when the object is not found. This makes WCET-based scheduling inefficient, while hard real-time guarantees are still required.

The objective of this paper is to explain the main principles of execution overrun handling and to propose a small software prototype that can be used to demonstrate the problem in a student project.

II. BACKGROUND

A. Hard Real-Time Tasks

A hard real-time task is a task that must complete before its deadline. A task can be represented as:

$$\tau_i(C_i, T_i, D_i)$$

where C_i is the execution time, T_i is the period, and D_i is the relative deadline.

For periodic tasks, the processor utilization of task τ_i is:

$$U_i = \frac{C_i}{T_i}$$

or, using frequency $f_i = 1/T_i$:

$$U_i = C_i f_i$$

The total processor utilization is:

$$U = \sum_{i=1}^n U_i$$

For Earliest Deadline First (EDF) scheduling on a single processor, a common schedulability condition is:

$$\sum_{i=1}^n U_i \leq 1$$

This means that the total computational demand must not exceed the available processor capacity.

B. WCET and Normal Execution Time

In classical hard real-time scheduling, the WCET is used to guarantee safety. For task τ_i , the worst-case execution time is denoted as:

$$WCET_i$$

However, the main paper distinguishes between WCET and normal computation time:

$$c_i^n$$

where c_i^n is the normal or typical execution time of the task.

An overrun occurs when the actual execution time c_i exceeds the normal budget:

$$c_i > c_i^n$$

The overrun amount is:

$$c_i - c_i^n$$

The problem is that c_i^n may be much smaller than $WCET_i$. Scheduling according to $WCET_i$ is safe but pessimistic, while scheduling according to c_i^n is efficient but unsafe unless overruns are handled.

C. Performance Loss Index

In control systems, task frequency affects control quality. Higher sampling frequencies usually improve responsiveness and reduce control error, but they also increase processor load.

The main paper uses the Performance Loss Index (PLI) to model the relationship between sampling frequency and control performance:

$$\Delta J_i(f_i) = \alpha_i e^{-\beta_i f_i}$$

where f_i is the task frequency, α_i is a magnitude coefficient, and β_i is a decay rate. The total system performance loss can be expressed as:

$$\Delta J(f_1, \dots, f_n) = \sum_{i=1}^n w_i \Delta J_i(f_i)$$

where w_i represents the relative importance of each task.

The scheduling problem is therefore not only to meet deadlines, but also to choose task frequencies that improve control performance while keeping the system schedulable.

III. HANDLING EXECUTION OVERRUNS

A. Problem with Uncontrolled Overruns

If a task overruns and continues executing without restriction, it may delay other tasks. In a hard real-time system, this can jeopardize previously computed timing guarantees. A single task with unexpected execution demand can therefore affect the whole task set.

The goal of overrun handling is to prevent one task from introducing unbounded interference into the system.

B. Local Overrun Handling

Caccamo et al. propose handling overruns locally. This means that an overrun should mainly affect the task that caused it, rather than disturbing all other tasks.

Each task is assigned a bandwidth:

$$U_i = f_i^{opt} c_i^n$$

where f_i^{opt} is the optimized frequency computed using normal execution time. If the task exceeds its normal computation time, its deadline can be postponed so that it does not exceed its reserved bandwidth.

The advantage is temporal isolation. The overrunning task may be delayed, but other tasks are protected from its extra execution demand.

C. Constant Bandwidth Server

The Constant Bandwidth Server (CBS) is a scheduling mechanism that provides bandwidth reservation. A CBS is defined by:

$$(Q_s, T_s)$$

where Q_s is the maximum budget and T_s is the server period. The server bandwidth is:

$$U_s = \frac{Q_s}{T_s}$$

When a job executes, the server budget is consumed. If the budget is exhausted, the server deadline is postponed and the budget is replenished. This reduces the priority of the task under EDF and prevents it from stealing unlimited processor time from other tasks.

The simplified CBS behavior can be written as:

if task executes :

decrease server budget

if server budget is exhausted :

recharge budget

postpone server deadline

CBS is useful because it allows a task with variable execution time to continue executing while still limiting its long-term processor usage.

D. CBShd

The main paper also discusses an improved CBS variant called CBShd. The motivation is that standard CBS can postpone a deadline too far when only a small remaining overrun exists. CBShd adjusts the budget recharge according to the remaining computation time.

A simplified CBShd rule is:

if remaining_time >= server_budget :

recharge full budget

postpone deadline by server period

else :

recharge only remaining_time

postpone deadline only as needed

This improves responsiveness because small overruns do not cause unnecessarily large deadline postponements.

IV. RESOURCE AND IMPLEMENTATION ISSUES

A. Runtime Overhead

Overrun handling is not free. The scheduler must manage budgets, deadlines, and replenishment events. Smaller server budgets can improve precision because the task is split into smaller chunks, but this increases the number of budget exhaustion events and deadline updates.

The main tradeoff is:

- small budget: better overrun precision, but higher overhead;
- large budget: lower overhead, but less precise overrun handling.

This is important for embedded systems where processor time is limited.

B. Resource Sharing

Real embedded tasks often share resources such as memory buffers, sensors, communication interfaces, or hardware peripherals. If an overrunning task holds a shared resource, other tasks may be blocked. Resource access protocols such as the Stack Resource Policy (SRP) can be used to bound blocking times.

For a five-page seminar paper, resource sharing can be discussed briefly, since the main focus is the handling of execution overruns.

V. PROTOTYPE SIMULATION FOR LEARNING PURPOSES

A. Purpose of the Prototype

A full kernel-level implementation of CBS or CBS_{hd} is beyond the scope of this seminar. Instead, a simplified software prototype can be used to demonstrate the key ideas:

- normal execution time,
- occasional overruns,
- WCET-based inefficiency,
- deadline misses without overrun handling,
- improvement using a simplified budget/deadline rule.

The prototype should be presented as a learning simulation, not as a real hard real-time scheduler.

B. Preferred Implementation: Python Image Processing

The easiest implementation is in Python using OpenCV or NumPy. The idea is to simulate a visual tracking task. An image contains a colored object, such as a red circle. The task first searches for the object in a small predicted window. If the object is not found, the search region is expanded. If the object is still not found, the full image is searched.

This creates different execution-time cases:

- easy case: object found in small predicted window;
- medium case: object found after expanding the search;
- hard case: object found only after full-image search;
- worst case: object missing, requiring full search.

Synthetic images are recommended because they allow controlled experiments. The object position can be chosen deliberately to create normal, overrun, and worst-case scenarios.

C. Prototype Architecture

The prototype can be organized into three programs:

- `generate_images.py`: creates synthetic images with easy, medium, hard, and missing-object cases;
- `overrun_demo.py`: processes each image and measures execution time;
- `scheduler_simulation.py`: compares different scheduling strategies.

A possible folder structure is:

```
implementation /
  generate_images.py
  overrun_demo.py
  scheduler_simulation.py

images /
  easy /
  medium /
  hard /
  missing /

results /
  runtime_log.csv
  runtime_plot.png
  deadline_misses.png
  strategy_comparison.png
```

D. Measured Values

For each image or frame, the program should log:

- frame number,
- difficulty level,
- search mode,
- execution time,
- normal budget,
- estimated WCET,
- deadline,
- overrun flag,
- deadline-miss flag,
- scheduling strategy.

The execution time can be measured using Python's high-resolution timer:

```
start = time.perf_counter()
# image processing task
finish = time.perf_counter()
execution_time_ms = (finish - start) * 1000
```

E. Scheduling Strategies to Compare

Three strategies can be compared.

1) *WCET-Based Scheduling*: Every job is scheduled using the worst observed execution time. This should produce few or no deadline misses, but low efficiency because the processor is reserved for rare worst-case behavior.

2) *Normal-Budget Scheduling*: Every job is scheduled using the normal execution time. This improves normal-case efficiency but can cause deadline misses when hard or missing-object cases occur.

3) *Simplified Overrun Handling*: A simplified CBS-inspired rule is used. Each task receives a normal budget. If the execution time exceeds this budget, the extra time is treated as an overrun and the deadline is adjusted according to a bandwidth rule.

Let:

$$U_s = \frac{\text{normal_budget}}{\text{period}}$$

If:

$$\text{execution_time} > \text{normal_budget}$$

then:

$$\text{extra} = \text{execution_time} - \text{normal_budget}$$

and the adjusted deadline extension is:

$$\text{deadline_extension} = \frac{\text{extra}}{U_s}$$

This is not a full CBS implementation, but it demonstrates the principle of budget-based overrun handling.

F. Prototype Pseudocode

The image-processing task can be described as:

```
for each image:
    start timer

    found = search_small_window(image)

    if not found:
        found = search_medium_window(image)

    if not found:
        found = search_full_image(image)

    stop timer
    compute execution time

    if execution_time > normal_budget:
        mark overrun
    else:
        no overrun

    log result
```

The simplified scheduler simulation can be described as:

```
for each measured execution time:
    if strategy == WCET:
        budget = wcet_estimate
```

```
if strategy == Normal:
    budget = normal_budget

if strategy == OverrunHandled:
    budget = normal_budget
    if execution_time > budget:
        extra = execution_time - budget
        deadline += extra / bandwidth

check deadline miss
log result
```

G. Possible UPPAAL Model

UPPAAL can also be used, but it is better for modeling and verification than for measuring real execution time. A UPPAAL model could represent the timing behavior abstractly.

Possible UPPAAL templates:

- **Task**: normal execution, overrun execution, completion;
- **Scheduler**: grants execution budget and checks deadlines;
- **Server**: models budget exhaustion and deadline postponement;
- **Environment**: chooses easy, medium, hard, or worst-case input.

Example properties to verify:

- no hard deadline is missed under the overrun-handling rule;
- an overrun can occur;
- the scheduler eventually returns to normal execution;
- a normal-budget-only strategy can lead to a deadline miss.

A possible UPPAAL-style query is:

$A[]$ not Task.deadline_missed

for the overrun-handled model, and:

$E\Diamond$ Task.deadline_missed

for the unsafe normal-budget-only model.

UPPAAL is useful for formal demonstration, but Python is easier for producing plots and measurable runtime data for the paper and presentation.

H. C++ Alternative

A C++ implementation is also possible and may give more consistent timing than Python. The structure would be similar:

- use OpenCV for image processing;
- use `std::chrono::high_resolution_clock` for timing;
- log results to CSV;
- analyze results using Python or spreadsheet software.

However, Python is recommended for the first prototype because it is faster to develop and easier to visualize.

VI. EXPECTED LEARNING OUTCOMES

The prototype is expected to demonstrate the following:

- WCET-based scheduling is safe but inefficient;
- normal-time scheduling improves average performance but can miss deadlines;
- overrun handling can limit the effect of exceptional long execution times;
- image-processing tasks are suitable examples because their execution time depends on input difficulty;
- practical timing measurements are affected by the operating system, hardware, and background processes.

Since the prototype is not a real-time kernel implementation, the results should be interpreted as an educational simulation rather than a formal hard real-time guarantee.

VII. DISCUSSION

The handling of execution overruns is important because many modern embedded systems perform computations with input-dependent execution times. Visual tracking, sensor fusion, and AI-based perception tasks may execute quickly in simple cases but require significantly more processing in difficult cases.

The approach proposed by Caccamo et al. is valuable because it does not simply choose between safety and performance. Instead, it uses normal execution time to improve control performance while using bandwidth reservation and deadline postponement to preserve schedulability under bounded worst-case conditions.

However, practical implementation remains challenging. WCET estimation is difficult on modern hardware due to caches, interrupts, operating system behavior, and input-dependent execution paths. In addition, server-based scheduling introduces runtime overhead, and resource sharing can complicate the analysis.

For a student seminar, a simplified prototype is useful because it makes the abstract scheduling concept visible. By showing runtime spikes, deadline misses, and the effect of a budget-based overrun rule, the prototype can support both the paper and the presentation.

VIII. CONCLUSION

Execution overruns occur when real-time tasks execute longer than their expected or reserved computation time. In hard real-time control systems, uncontrolled overruns can break schedulability guarantees and cause deadline misses. Traditional WCET-based scheduling is safe but can waste processor bandwidth when worst-case behavior is rare.

Rate adaptation and server-based scheduling provide a more efficient solution. The Constant Bandwidth Server limits task interference using budget and deadline mechanisms, while CBS_{hd} improves the handling of hard-deadline periodic tasks by avoiding unnecessary deadline postponements for small overruns.

A proposed Python/OpenCV prototype can demonstrate these ideas using a simplified image-processing task. The prototype compares WCET-based scheduling, normal-budget

scheduling, and simplified overrun handling. Although it does not provide kernel-level hard real-time guarantees, it offers a practical learning tool for visualizing execution-time variation and the purpose of overrun management.

REFERENCES

- [1] M. Caccamo, G. Buttazzo, and L. Sha, "Handling Execution Overruns in Hard Real-Time Control Systems," *IEEE Transactions on Computers*, vol. 51, no. 7, pp. 835–849, 2002.
- [2] G. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 4th ed. Springer, 2024.
- [3] L. Abeni and G. Buttazzo, "Integrating Multimedia Applications in Hard Real-Time Systems," in *Proceedings of the IEEE Real-Time Systems Symposium*, 1998.
- [4] C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.
- [5] D. Seto, J. P. Lehoczky, L. Sha, and K. G. Shin, "On Task Schedulability in Real-Time Control Systems," in *Proceedings of the IEEE Real-Time Systems Symposium*, 1996.
- [6] T. P. Baker, "Stack-Based Scheduling of Real-Time Processes," *Real-Time Systems*, vol. 3, no. 1, pp. 67–99, 1991.
- [7] G. Bradski, "The OpenCV Library," *Dr. Dobbs's Journal of Software Tools*, 2000.